

编译原理大作业第一部分（词法分析）报告

学号：524031910374

姓名：曾清

本次实验在 Pony 编译器框架的基础上，完成了词法分析器（Lexer）的核心功能实现。具体而言，需要实现三个函数：Lexer.h 中的 getNextChar() 与 getTok()，以及 ponyc.cpp 中的 dumpToken()。前两者负责从源文件中逐字符读取并识别各类 Token，后者负责遍历词法分析结果并将其输出到终端，以验证词法分析器的正确性。

本报告共分为四个部分：

1. getNextChar() 函数的实现思路与方法
2. getTok() 函数的实现思路与方法
3. dumpToken() 函数的实现思路与方法
4. 实验结果展示与分析

一、getNextChar() 的实现

功能描述

getNextChar() 负责从当前行缓冲区 curLineBuffer 中逐字符读取内容，并在当前行读取完毕后自动调用 readNextLine() 加载下一行，同时维护行列位置信息。

实现思路

整体逻辑分为三步：

第一步：处理缓冲区为空的边界情况。 若 curLineBuffer 为空，说明已到达文件末尾，直接返回 EOF。

第二步：读取当前字符并推进缓冲区。 使用 front() 取出队首字符，再用 drop_front() 将其从缓冲区中移除。这里需要注意 llvm::StringRef 的 drop_front() 会返回一个新的 StringRef 而不修改原对象，因此必须将返回值重新赋给 curLineBuffer（我开始时就是以为它是修改原对象的，直接用了 curLineBuffer.drop_front()；导致测试时没有输出）：

```
char nextChar = curLineBuffer.front();
curLineBuffer = curLineBuffer.drop_front(); // 必须赋值，否则缓冲区不会前进
```

第三步：同步更新行列信息。 若读到换行符 '\n'，则调用 readNextLine() 读入下一行，行号加一，列号归零；否则列号自增：

```
if (nextChar == '\n') {
    curLineBuffer = readNextLine();
    curLineNum++;
    curCol = 0;
}
```

```
    } else {  
        curCol++;  
    }  
    return nextChar;  
}
```

初始状态下 `curLineBuffer = "\n"`，第一次调用 `getNextChar()` 会读到 `'\n'` 并触发 `readNextLine()` 加载文件第一行。

二、`getTok()` 的实现

功能描述

`getTok()` 在 `getNextChar()` 的基础上，将字符流归并为有意义的 Token，包括关键字、标识符、数字、符号及注释，并对非法标识符和非法数字进行检测与报告。

2.1 关键字与标识符的识别

当首字符为字母或下划线时，进入标识符识别分支。循环读取后续的字母、数字或下划线，拼接为完整字符串：

```
if (isalpha(lastChar) || lastChar == '_') {  
    std::string idStr;  
    do {  
        idStr += lastChar;  
        lastChar = Token(getNextChar());  
    } while (isalnum(lastChar) || lastChar == '_');  
    ...  
}
```

关键字匹配：为了兼容大小写不规范的书写形式（如 `Return`、`DEF`），先将读取到的字符串全部转换为小写，再与 `"return"`、`"def"`、`"var"` 进行比较，匹配则返回对应的关键字 Token。

非法标识符检测：根据题目规则，标识符中的数字不可连续出现。对此，用 `lastIsNum` 标志位在遍历字符时追踪上一个字符是否为数字，若连续出现两个数字则标记为非法，输出错误信息并将 `valid` 置为 `false`：

```
bool lastIsNum = false;  
for (char c : idStr) {  
    if (!isdigit(c)) { lastIsNum = false; continue; }  
    if (lastIsNum) { invalid = true; break; }  
    lastIsNum = true;  
}
```

2.2 数字的识别与非法检测

当首字符为数字或 `.` 时进入数字识别分支。为了能捕获 `9e01` 这类含 `e` 的非法表示，循环条件也纳入了 `'e'`：

```
while (isdigit(lastChar) || lastChar == '.' || lastChar == 'e')
```

合法性检测：用状态标志 `haveDot`、`haveE`、`lastIsDot`、`lastIsE` 对数字字符串进行一次遍历，覆盖以下几类非法情形：

非法形式示例	触发条件
9.9.9	<code>haveDot</code> 为真时再次出现 .
.e9	<code>lastIsDot</code> 为真，后面紧跟的不是数字
9e01	<code>lastIsE</code> 为真，后面紧跟的不是 1~9
9ee1	<code>haveE</code> 为真时再次出现 e

检测到非法时，输出错误信息并将 `valid` 置为 `false`；否则置为 `true`，并调用 `strtod` 解析数值。

2.3 valid 标志位的设计

为了让 `dumpToken()` 能够判断当前 Token 是否有效、从而决定是否将其输出，在 `Lexer` 类中新增了私有成员变量 `valid` 和对应的公有访问方法 `getValid()`。每次识别标识符或数字时，根据合法性结果同步更新 `valid`。

三、dumpToken() 的实现

功能描述

`dumpToken()` 初始化词法分析器，遍历源文件中的所有 Token，并按顺序输出到终端，最终输出 `EOF` 表示文件结束。

实现思路

初始化 `LexerBuffer` 后，调用 `getNextToken()` 完成预读（prime），随后进入循环：

```
lexer.getNextToken(); // prime the lexer
while (lexer.getCurToken() != tok_eof) {
    Token curToken = lexer.getCurToken();
    ...
    lexer.consume(curToken);
}
std::cout << "EOF\n";
```

Token 输出：对于关键字直接输出字符串，对于标识符和数字，先通过 `lexer.getValid()` 检查有效性，有效才输出；对于其他符号，将其 ASCII 值强转为 `char` 输出：

```
} else if (curToken == tok_identifier) {
    if (lexer.getValid()) std::cout << lexer.getId().str() << ' ';
```

```
    } else if (curToken == tok_number) {  
        if (lexer.isValid()) std::cout << lexer.getValue() << ' ';  
    } else {  
        std::cout << (char)curToken << ' ';  
    }  
}
```

非法的标识符和数字由 Lexer 内部通过 `std::cerr` 输出错误信息，`dumpToken()` 只负责过滤，不重复输出。这样实现了错误信息与正常 Token 流的分离，结果清晰。

四、实验结果展示与分析

完成代码的补充和编译后，我用 `test_1.pony` ~ `test_7.pony` 进行了测试，均能够顺利完成词法分析、在遇到错误时正确报错。现在选取 `test_1.pony` , `test_6.pony` , `test_7.pony` 作为样例在报告中进行检测和分析，它们分别代表了无报错、含有无效数字、含有无效标识符三种情况。

1. `test_1.pony`

`test_1.pony` 源代码如下：

```
# ../build/bin/pony ../test/test_1.pony -emit=token  
  
def main() {  
    Var a[2][3] = [1, 2, 3, 4, 5, 6];  
}
```

经过词法分析输出的结果为：

```
def main ( ) { var a [ 2 ] [ 3 ] = [ 1 , 2 , 3 , 4 , 5 , 6 ] ; } EOF
```

对比源代码和输出结果可以看到，成功完成了词法分析，正确识别了大小写不规范的 `Var` 关键字并输出为全小写形式 `var`。

2. `test_6.pony`

`test_6.pony` 源代码如下：

```
# ../build/bin/pony ../test/test_6.pony -emit=token  
  
def main() {  
    var a[2][3] = [1, 2.3., 3, 4e03, 5, 6];  
}
```

```
}
```

经过词法分析输出的结果为：

```
def main ( ) { var a [ 2 ] [ 3 ] = [ 1 ,  
line 6, col 22:  
ERROR: Invalid number: 2.3.  
, 3 ,  
line 6, col 31:  
ERROR: Invalid number: 4e03  
, 5 , 6 ] ; } EOF
```

源码中有两处无效数字，分别是 `2.3.`，`4e03`，从词法分析结果可以看出这两处错误都被准确找到并指出了错误所在的行号、列号和错误原因。

3. test_7.pony

`test_7.pony` 源代码如下：

```
# ../build/bin/pony ../test/test_7.pony -emit=token  
  
def main() {  
  
    var a16[2][3] = [1, .23, 3.2e7, 4, 5, 6];  
  
}
```

经过词法分析输出的结果为：

```
def main ( ) { var  
line 5, col 8:  
ERROR: Invalid identifier with continuous numbers: a16  
[ 2 ] [ 3 ] = [ 1 , 0.23 , 3.2e+07 , 4 , 5 , 6 ] ; } EOF
```

源码中有一处无效标识符 `a16`，因为出现了连续数字，由词法分析结果可以看到这一错误被准确定位并指明错误原因。同时，由输出的 `0.23`，`3.2e+07` 也可以看到词法分析正确地将源码中的数字转换到了 `double` 类型。